

AI LAB MANUAL

Twenty-five experiments in machine learning and heat transfer

A companion to Heat Transfer: Energy in Transit

ME 313

Prasoon Diwakar

South Dakota School of Mines & Technology

Fall 2026

Contents

Before you start	iii
1 Introduction & Steady-State Conduction	1
2 The Heat Diffusion Equation & Generation	3
3 Extended Surfaces (Fins)	5
4 Transient Conduction	7
5 External Flow Convection	9
6 Internal Flow Convection & LMTD	11
7 Radiation	13
Lab index	15

Before you start

What these labs are

Twenty-five short experiments, one set per module, that put machine learning next to the physics you are already learning. They are optional enrichment: nothing here replaces the course material, and every lab ends by *checking the computer against the physics*. That check is the point. You do not need prior machine-learning experience, and you do not need to install anything.

The four kinds of lab

Every module offers the same ladder, so once you have done one module's set you know the shape of all the others:

- **X.0 — Audit.** Ask an AI to solve a problem, then catch its mistakes with the physics. No programming, no ML background. *Start here.*
- **X.1 — Data fit.** Recover a physical law or property from data (a correlation, a convection coefficient, a temperature).
- **X.2 — Surrogate or PINN.** The showpiece: a model that replaces an expensive calculation, or a network that solves the governing equation itself.
- **X.3 — Design.** Point the tool at a target and let it search for a good design.

Getting started with Google Colab

Colab runs Python in your browser on Google's computers. Nothing installs on your machine, and it is free with a Google account.

1. **Open the notebook.** Download the lab's .ipynb file from the course page, then go to `colab.research.google` and choose **File → Upload notebook**. (If a lab is posted as a link, just click it.)
2. **Save your own copy first.** **File → Save a copy in Drive**. Otherwise your edits vanish when you close the tab. The copy is yours to break.
3. **Run cells in order, top to bottom.** Click a cell and press **Shift + Enter**. A cell that has run shows a number in brackets beside it. Running out of order is the single most common cause of confusing errors.
4. **If something breaks, restart and rerun.** **Runtime → Restart and run all** fixes most problems.
5. **Only if a package is missing,** add a cell with, for example, `!pip install torch` and run it. Most labs need nothing extra — Colab already has NumPy, SciPy, scikit-learn, Matplotlib and PyTorch.

6. **For the PINN labs**, switch on a GPU if training feels slow: **Runtime** → **Change runtime type** → **GPU**. A CPU still works; it is just slower.
7. **To hand work in**, use **File** → **Print** → **Save as PDF**, or **File** → **Download** → **.ipynb**, and submit through D2L.

Read the error's last line first — it names the actual problem. **Change one thing at a time** when a result looks wrong. **Sanity-check every number against the physics** — if a fin efficiency comes back as 1.4 or a temperature as -40°C in a furnace, the code is wrong, not the physics.

The verification habit

Every lab ends the same way: compare the model against something you already trust — an analytical solution, a textbook correlation, a limiting case, or a unit check. Three questions catch most errors, whether they come from an AI, a neural network, or your own algebra:

1. Are the **units** consistent throughout?
2. Is the **magnitude** physically sensible?
3. Do the **assumptions** behind the formula actually hold in this problem?

In the audit labs you are *supposed* to have an AI solve the problem — that is the assignment. Everywhere else, the usual course rules apply: do your own work first, and make clear what came from a tool. Auditing an AI is a skill worth having precisely because these tools are confidently wrong often enough to matter.

MODULE 1

Introduction & Steady-State Conduction

Ready-to-run notebook for this module: Lab 1.1

Low - ~45 min

Goal. build the habit of auditing AI answers with physics.

The idea. LLMs as fallible problem-solvers; verification as a skill.

You do.

1. ask an LLM to fully solve a composite-wall or insulated-pipe problem
2. rebuild its answer as a thermal-resistance network by hand
3. find and explain every discrepancy (sign, area, units)

Verify with. the unit check and $Q=\Delta T/R_{\text{total}}$ from the notes.

Hand in. a short error log (what the AI got wrong and why the physics caught it) plus the corrected solution.

Build your own notebook — start from a provided one in this module and adapt it.

Low-Med - ~75 min

Goal. see why h is empirical by fitting it from data.

The idea. regression / a small neural net mapping flow features to h or Nu .

You do.

1. fit a linear regression on a provided
2. dataset
3. fit a small neural net and compare
4. test both against a textbook correlation on held-out data

Verify with. a known Nu correlation on the held-out points.

Hand in. a notebook, a parity plot, an error metric, and a paragraph on why h resists a single universal formula.

Notebook provided: `ME313_Lab1_1_learn_h_from_data.ipynb`

Low - ~60 min

Goal. understand what a surrogate model buys and where it fails.

The idea. regression surrogate of a quantity whose ground truth is exact.

You do.

1. sample the analytical plane-wall $Q=kA\Delta T/L$ over many inputs
2. train a surrogate to predict
3. probe it inside and outside the training range

Verify with. the analytical Q — the truth is known everywhere.

Hand in. a parity plot and a short note on accuracy inside vs. outside the training envelope (extrapolation risk).

Build your own notebook — start from a provided one in this module and adapt it.

MODULE 2

The Heat Diffusion Equation & Generation

Ready-to-run notebook for this module: Lab 2.1

Low - ~45 min

Goal. catch the classic generation-problem mistakes.

The idea. LLM as a setup generator; the energy balance as the referee.

You do.

1. have an LLM set up a plane-wall-with-generation problem and its BCs
2. solve it yourself
3. confirm the energy-closure check holds and flag any AI error

Verify with. the three-flux $q_{\text{gen}}=q_{\text{conv}}=q_{\text{cond}}$ closure.

Hand in. the verified setup plus an error log of any AI slips.

Build your own notebook — start from a provided one in this module and adapt it.

Medium - ~90 min

Goal. solve a heat-generation problem with a neural network and trust it because it matches the analytical answer.

The idea. a physics-informed neural network for $d^2T/dx^2 + \dot{g}/k = 0$.

You do.

1. fill the physics into a scaffolded PINN notebook (loss = PDE residual + BCs)
2. train on the insulated-back / cooled-face bus-bar
3. overlay the PINN profile on the analytical $T(x)=405-2000x^2$

Verify with. the analytical Module 2 solution — report the maximum error.

Hand in. the PINN solution, an overlay plot, and the peak error vs. the exact result.

Notebook provided: `ME313_Lab2_1_PINN_busbar.ipynb`

Medium - ~90 min

Goal. infer an unknown $\dot{g}$ from sparse temperature data — an inverse problem.

The idea. data assimilation: combine a few sensor readings with the governing PDE.

You do.

1. take a handful of synthetic ‘sensor’ temperatures from the bus-bar
2. let a PINN or optimizer recover the unknown $\dot{g}$ (or a boundary condition)
3. compare the recovered value to the truth

Verify with. the known $\dot{g}$ used to generate the data.

Hand in. the inferred $\dot{g}$ with a short note on how sensor count and noise affect the estimate.

Build your own notebook — start from a provided one in this module and adapt it.

MODULE 3

Extended Surfaces (Fins)

Ready-to-run notebook for this module: Lab 3.1

Low - ~45 min

Goal. catch the effectiveness/efficiency and corrected-length traps.

The idea. LLM as solver; the fin checks as referee.

You do.

1. have an LLM solve a rectangular- or pin-fin problem
2. audit mL for dimensionlessness, the use of the corrected length, and ϵ vs. η
3. correct any error and explain it

Verify with. the dimensionless-mL check and the $\epsilon(A_c)$ vs. $\eta(A_f)$ distinction.

Hand in. a corrected solution and a short error log.

Build your own notebook — start from a provided one in this module and adapt it.

Low-Med - ~75 min

Goal. replace the fin-efficiency chart with a trained model.

The idea. a surrogate of $\eta_f = \tanh(mL_c)/(mL_c)$ — exact ground truth.

You do.

1. generate ($mL_c \rightarrow \eta_f$) data from the analytical formula
2. train a surrogate and use it to size a fin for a target Q_f
3. compare the sized fin to the analytical result

Verify with. the analytical η_f and $Q_f = \eta_f h A_f \theta_b$.

Hand in. a surrogate, a parity plot, and the sized-fin check against the exact answer.

Notebook provided: `ME313_Lab3_1_fin_efficiency_surrogate.ipynb`

Medium - ~90 min

Goal. use optimization to design, then sanity-check against intuition.

The idea. constrained optimization of total Q_f (or Q_f per unit mass).

You do.

1. fix base area and material; let an optimizer choose fin count, thickness and length
2. maximize total heat (or heat per mass) under a mass/spacing constraint
3. compare the optimal design to the diminishing-returns rule $mL \approx 2.3$

Verify with. the fin equations and the $mL \approx 2.3$ saturation insight from the notes.

Hand in. the optimized geometry, a trade-off plot, and a reflection linking the result to generative heat-exchanger design.

Build your own notebook — start from a provided one in this module and adapt it.

MODULE 4

Transient Conduction

Ready-to-run notebooks for this module: Lab 4.1, Lab 4.2

Low - ~45 min

Goal. catch the characteristic-length and criterion mistakes AI makes.

The idea. LLM as solver; the two-Biot rule and the Bi/Fo gates as referee.

You do.

1. have an LLM solve a quenched-cylinder or -sphere transient problem
2. check which L_c it used for Bi, whether it ran the $Bi \leq 0.1$ check, and whether $Fo > 0.2$
3. correct any error and explain it in one line

Verify with. the two-Biot rule and the $Bi \leq 0.1$ / $Fo > 0.2$ gates from the notes.

Hand in. a corrected solution plus a short error log.

Build your own notebook — start from a provided one in this module and adapt it.

Low-Med - ~75 min

Goal. extract a physical parameter from transient data — an inverse problem you can do with a fit.

The idea. nonlinear curve-fit of an exponential decay to recover the time constant.

You do.

1. load a (noisy) measured cooling curve of a small lumped body
2. fit $\theta/\theta_i = \exp(-t/\tau)$ to find τ
3. back out h (or α) from τ , and compare to the known value

Verify with. the lumped relation $\tau = \rho V c / h A_s$ and the value used to generate the data.

Hand in. the fitted τ , the recovered h , and a plot of fit vs. data.

Notebook provided: `ME313_Lab4_1_cooling_curve_fit.ipynb`

Medium - ~90 min

Goal. solve a time-dependent conduction problem with a neural network and trust it against the one-term answer.

The idea. a physics-informed neural network for the transient heat equation.

You do.

1. fill the physics into a scaffolded transient-PINN notebook (loss = PDE residual + IC + BCs)
2. train it on a cooling plane wall
3. compare the center-temperature history to the one-term series result

Verify with. the Module 4 one-term solution $\theta^* = C_1 e^{-\zeta_1^2 Fo}$.

Hand in. the PINN temperature history overlaid on the one-term curve, with the error.

Notebook provided: ME313_Lab4_2_transient_PINN.ipynb

Medium · ~90 min

Goal. build a fast surrogate that predicts a component's transient temperature.

The idea. a reduced-order / surrogate model trained on transient responses.

You do.

1. generate transient cooling responses across a range of loads and h values
2. train a surrogate to predict the temperature history
3. use it to check whether a proposed cooling setting keeps a chip under its limit

Verify with. the lumped/one-term response on held-out cases.

Hand in. the surrogate, a validation plot, and a go/no-go on the cooling setting.

Build your own notebook — start from a provided one in this module and adapt it.

External Flow Convection

Low · ~45 min

Goal. catch the correlation, length, and property-temperature mistakes AI makes.

The idea. LLM as solver; the six-step workflow as referee.

You do.

1. have an LLM solve a plate or cylinder convection problem
2. check the property temperature, the characteristic length, the correlation and its validity range
3. correct any error and explain it in one line

Verify with. the six-step workflow and correlation validity ranges from the notes.

Hand in. a corrected solution plus a short error log.

Build your own notebook — start from a provided one in this module and adapt it.

Low-Med · ~75 min

Goal. recover a $Nu = C Re^m Pr^n$ correlation from data — see that the textbook fit is itself machine-learnable.

The idea. log-space linear regression (and a small neural net) on a $Nu(Re, Pr)$ dataset.

You do.

1. load a (noisy) dataset of Nu, Re, Pr
2. fit $\log Nu = \log C + m \log Re + n \log Pr$ to recover the exponents
3. compare the recovered correlation to a textbook one

Verify with. a known correlation (e.g. the flat-plate or Dittus-Boelter exponents).

Hand in. the recovered C, m, n and a parity plot.

Build your own notebook — start from a provided one in this module and adapt it.

Medium · ~90 min

Goal. predict h (or the local Nu_x) from a fast model instead of a full simulation.

The idea. a surrogate trained on correlation/CFD data, or a PINN for the boundary-layer equations.

You do.

1. generate h (or Nu_x) across a range of Re and Pr from correlations
2. train a surrogate to predict it
3. test it on held-out conditions and time the speed-up

Verify with. the Module 5 correlations on held-out cases.

Hand in. a validation plot and the surrogate's speed vs. accuracy trade-off.

Build your own notebook — start from a provided one in this module and adapt it.

Medium · ~90 min

Goal. choose flow speed and geometry to hit a target h and keep a component under its limit.

The idea. a surrogate/optimizer over the convection correlations.

You do.

1. express component temperature as a function of air speed and geometry via the correlations
2. search for settings that meet a temperature limit at least fan power
3. verify the chosen design against the direct correlation

Verify with. the six-step workflow on the final design point.

Hand in. a recommended cooling setting with its predicted h and margin.

Build your own notebook — start from a provided one in this module and adapt it.

MODULE 6

Internal Flow Convection & LMTD

Ready-to-run notebook for this module: Lab 6.2

Low - ~45 min

Goal. catch the LMTD-validity and correlation mistakes AI makes on exchanger problems.

The idea. LLM as solver; the four LMTD conditions and the internal/external correlation split as referee.

You do.

1. have an LLM solve the composite-cylinder problem (h_i , h_o , U , Q)
2. check the correlation choice for each h , the Dittus-Boelter exponent, and the four LMTD conditions
3. correct any error and explain it in one line

Verify with. the Module 6 overall- U equation and LMTD-validity list.

Hand in. a corrected solution plus a short error log.

Build your own notebook — start from a provided one in this module and adapt it.

Low-Med - ~75 min

Goal. recover a Dittus-Boelter-type correlation from data and see its limits.

The idea. log-space regression on a $Nu(Re, Pr)$ pipe-flow dataset.

You do.

1. load a (noisy) internal-flow dataset of Nu , Re , Pr
2. fit $\log Nu = \log C + a \log Re + b \log Pr$ to recover the exponents
3. compare to Dittus-Boelter (0.023, 0.8, 0.4/0)

Verify with. the Dittus-Boelter exponents.

Hand in. the recovered C , a , b and a parity plot.

Build your own notebook — start from a provided one in this module and adapt it.

Medium - ~90 min

Goal. predict outlet temperature (or UA) across operating conditions without re-solving LMTD each time.

The idea. a surrogate trained on LMTD/energy-balance solutions.

You do.

1. generate exchanger outputs across flow rates and inlet temperatures via the Module 6 equations
2. train a surrogate to predict the outlet temperature

3. test on held-out conditions and time the speed-up

Verify with. the direct LMTD/energy-balance solution on held-out cases.

Hand in. a validation plot and the surrogate's accuracy vs. speed.

Notebook provided: `ME313_Lab6_2_exchanger_surrogate.ipynb`

Medium · ~90 min

Goal. choose tube size and flow rate to meet a heat duty at least pumping power.

The idea. surrogate + optimizer over the internal-flow correlations and overall-U.

You do.

1. express duty and pressure drop as functions of D and flow rate via the correlations
2. search for designs meeting the duty at minimum pumping power
3. verify the chosen design against the direct calculation

Verify with. the six-step / overall-U workflow on the final design.

Hand in. a recommended design point with its predicted UA and margin.

Build your own notebook — start from a provided one in this module and adapt it.

MODULE 7

Radiation

Ready-to-run notebooks for this module: Lab 7.1, Lab 7.2

Low · ~45 min

Goal. catch the kelvin, fourth-power, and α -vs- ϵ mistakes AI makes.

The idea. LLM as solver; the three-check habit as referee.

You do.

1. have an LLM solve an enclosure-exchange or greenhouse problem
2. check kelvin, T^4 difference (not $(dT)^4$)
3. , and whether α/ϵ were kept distinct
4. correct any error and explain it in one line

Verify with. the Module 7 formulas and the α -vs- ϵ (car) lesson.

Hand in. a corrected solution plus a short error log.

Build your own notebook — start from a provided one in this module and adapt it.

Low-Med · ~75 min

Goal. recover a surface temperature from its emission spectrum — a mini remote-sensing problem.

The idea. nonlinear fit of the Planck curve (or Wien's peak) to spectral data.

You do.

1. load a (noisy) emission spectrum
2. fit the Planck distribution (or locate the peak and apply Wien) to find T
3. compare to the true temperature

Verify with. Wien's law and the Stefan-Boltzmann integral.

Hand in. the recovered temperature and a fit-vs-data plot.

Notebook provided: `ME313_Lab7_1_planck_temperature_fit.ipynb`

Medium · ~90 min

Goal. infer true temperature from a thermal image when emissivity is unknown or varies.

The idea. a small model that corrects apparent temperature for emissivity.

You do.

1. generate apparent vs. true temperatures across a range of emissivities
2. train a corrector that recovers true temperature
3. test it on held-out surfaces

Verify with. the $\epsilon\sigma T^4$ relationship on held-out cases.

Hand in. a validation plot of corrected vs. true temperature.

Notebook provided: ME313_Lab7_2_IR_emissivity_corrector.ipynb

Medium · ~90 min

Goal. choose surface properties to maximize solar gain or radiative cooling — the car problem, inverted.

The idea. optimization over spectral α and ϵ against a performance target.

You do.

1. express net absorbed/emitted power in terms of spectral properties and the two source spectra
2. search for property profiles that meet a target (e.g. hot solar absorber, or cool roof)
3. verify against the direct radiation balance

Verify with. the Module 7 exchange balance.

Hand in. a recommended property profile and its predicted performance.

Build your own notebook — start from a provided one in this module and adapt it.

Lab index

All twenty-five labs at a glance. “Provided” means a ready-to-run Colab notebook is posted on the course page; the others are yours to build, usually by adapting a provided notebook from the same module.

Lab	Module	Title	Notebook
1.0	1	Trust, but verify	build it
1.1	1	Learn the convection coefficient	provided
1.2	1	Surrogate vs. the exact answer	build it
2.0	2	Audit an AI derivation	build it
2.1	2	PINN the bus-bar	provided
2.2	2	Find the hidden source	build it
3.0	3	Fin-problem auditor	build it
3.1	3	Learn the efficiency surface	provided
3.2	3	Optimize a heat sink	build it
4.0	4	Transient-solution auditor	build it
4.1	4	Read h (or α) off a cooling curve	provided
4.2	4	PINN a cooling wall (or a moving heat source)	provided
4.3	4	A data-center thermal digital twin (lite)	build it
5.0	5	Convection-solution auditor	build it
5.1	5	Learn a Nusselt correlation from data	build it
5.2	5	A CFD surrogate (or boundary-layer PINN)	build it
5.3	5	Design a data-center cooling scheme	build it
6.0	6	Heat-exchanger solution auditor	build it
6.1	6	Fit a Nusselt (or friction) correlation from pipe data	build it
6.2	6	A heat-exchanger surrogate	provided
6.3	6	Optimize an exchanger design	build it
7.0	7	Radiation-solution auditor	build it
7.1	7	Fit temperature from a thermal spectrum (Wien/Planck)	provided
7.2	7	An IR-thermography emissivity corrector	provided
7.3	7	Design a spectrally selective coating	build it

Where to find everything

Notebooks, data files and any updates live on the course page under the **Teaching** tab: prasoondiwakar.github.io/plasma-lab. The Study Assistant described in the main book can help you think through any of these labs — including debugging your reasoning when a result looks wrong.